



The Making of an IO

A photograph of a man and a woman sitting at a restaurant table. The man, on the left, is gesturing with his hands while speaking. A speech bubble originates from him, containing text about Haskell and Phil Wadler. The woman, on the right, is listening. In the bottom right corner, there is a cutout of a man in a Superman costume. The table is set with white linens, glassware, and cutlery.

*The IO monad is
boilerplate Haskell
makes you use to
appease Phil Wadler.*

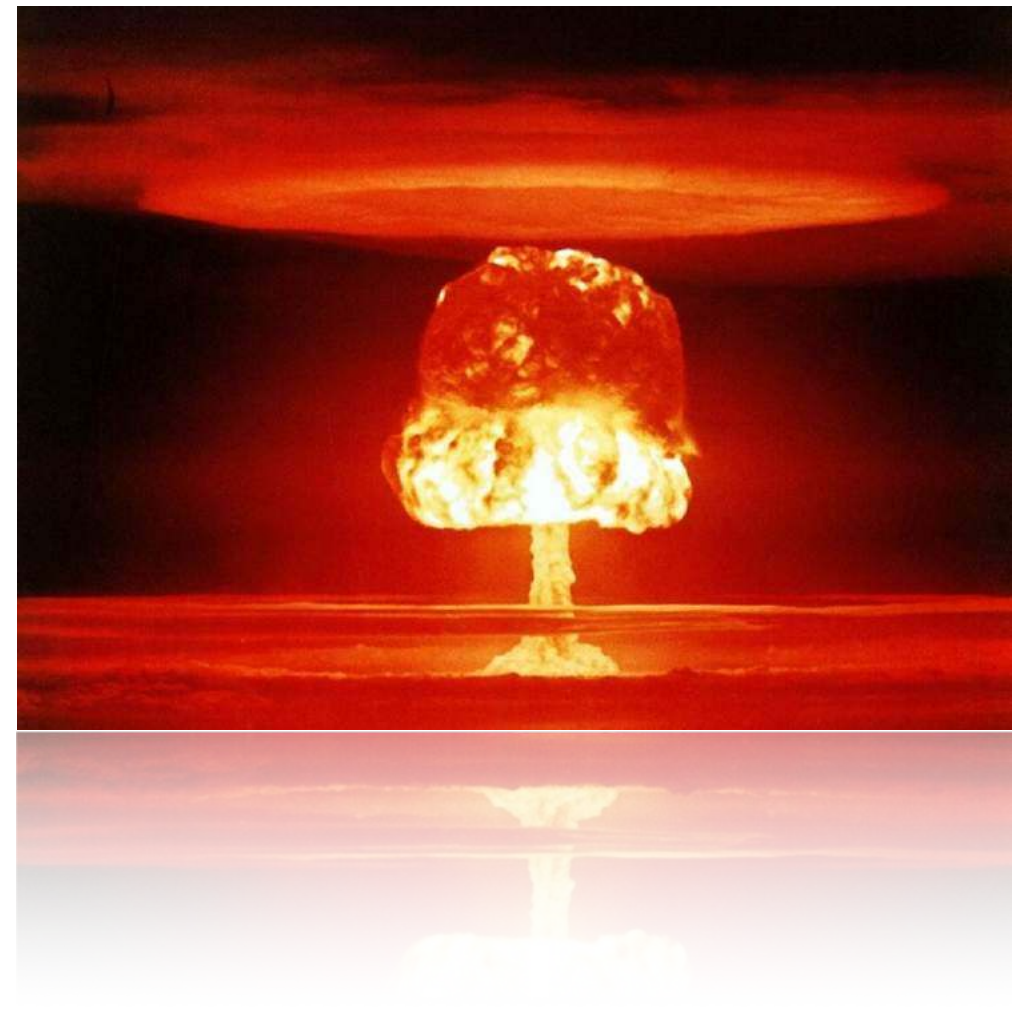


EFFECTS

HHD
HISTORY.COM

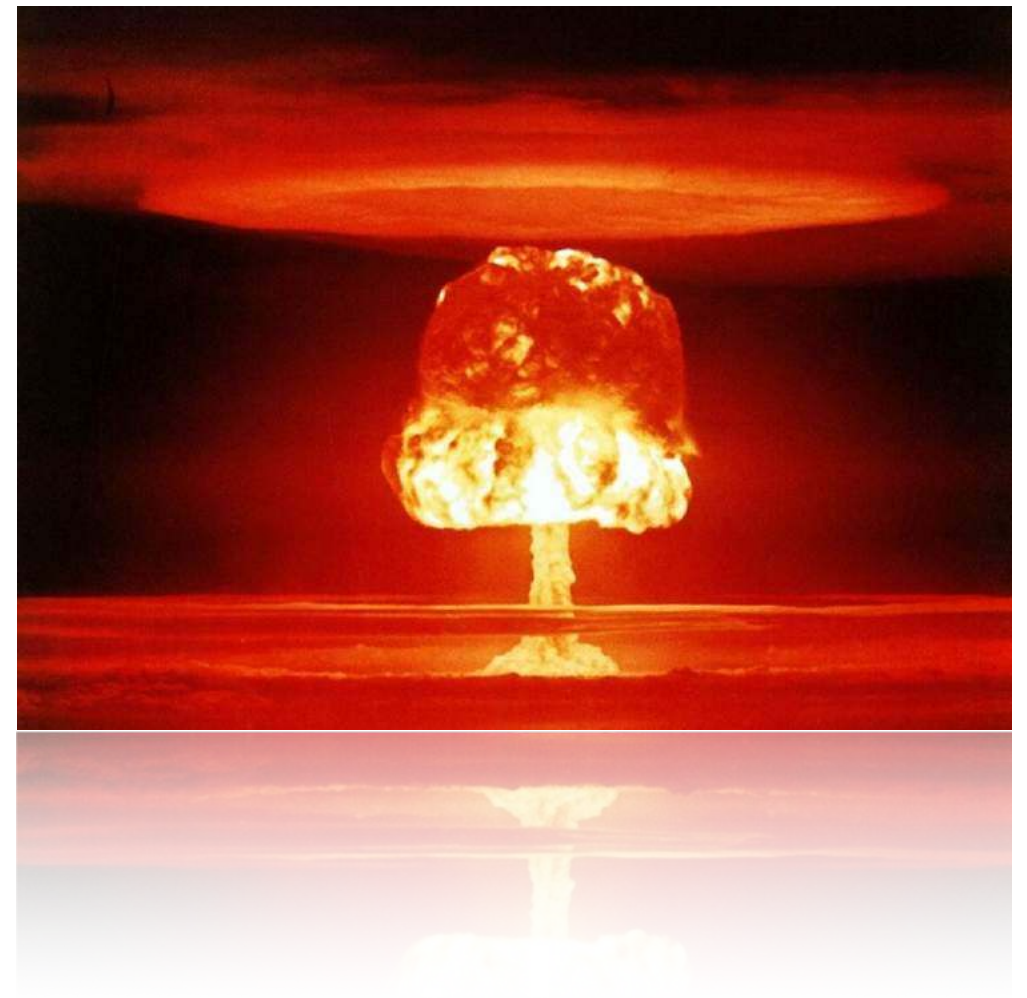
Effects

- Anything that you can't "just" do twice
- Writing to a database
- Reading from stdin
- Incrementing a counter
- Changing state in-place



Effects

- Are hard to test
- Make concurrency hard
 - ...and simultaneously useful
- Make refactoring hard



Effects

foo(42) + foo(42)

val x = foo(42)

x + x

Effects

addToGauge(42) + addToGauge(42)

```
val x = addToGauge(42)  
x + x
```

Effects

...should be controlled

Effects

```
def addToGauge(i: Int): IO[Int] = ???
```

```
for {  
  a <- addToGauge(42)  
  b <- addToGauge(42)  
} yield a + b
```

```
val x = addToGauge(42)
```

```
for {  
  a <- x  
  b <- x  
} yield a + b
```

The IO Monad

- Allows you to safely manipulate effects
- Build a *description* of the program to be run
 - Descriptions can be changed!
- Separate the composition from the declaration
 - Concurrency
 - Dynamic scaling



...monad?

- Program actions need to be sequentialized
- Monads are the *essence* of sequencing
 - This is literally the definition!
- We need a monad to say " A happens before B "
- This was Philip Wadler's insight




```
trait Monad[F[_]] {
```

```
  def pure[A](a: A): F[A]
```

```
  def flatMap[A, B](fa: F[A])(f: A => F[B]): F[B]
```

```
}
```

parametricity

Haskell

```
main :: IO ()  
main = putStrLn "Hello, World!"
```

Haskell

- All roads lead to IO
- All FFI is encapsulated within IO
- You cannot construct a new IO except with return
- Concurrency is managed by IO
 - ...sort of
 - Green threading

Scala

```
def main(args: Array[String]): Unit =  
  println("Hello, World!")
```



Scala

- The main method returns Unit
 - Meaning it runs side-effects and nothing else!
- There is no FFI between Scala and Java
- We can run any effects we want, at any time!
- Eager-by-default evaluation
- Low-level concurrency primitives

Scala

- Runs on the JVM!
- Native threads
 - Explicit asynchronous IO
 - Cannot block things!
- No tail-call elimination

IO in Scala

- Still a monad!
- Stack safety
- Different evaluation modes
 - Strict (for performance)
 - Lazy and synchronous
 - Asynchronous (callbacks)
- Effect-capturing constructors

Possibilities

Possibility

Possibility

Possibility

Possibility

Possibility



Future

- Runs actions eagerly by default
- Mer `val x: Future[Int] = addToGauge(42)` ple times
- A `for {` leaks memory
 - `a <- x`
- Doe `b <- x` capsule asynchronous execution well
 - `} yield a + b`
- Promise is leaky
- Rewrites exceptions into wrapped thingies

Scalaz 7 IO

- Doesn't support asynchronous at all!
- Encourages thread blocking and other horrors
 - MVar
- Only defines a lazy effect-capturing constructor
 - *Very* slow for pure computation!
- Provides zero abstraction beyond LiftIO

Scalaz 7 IO

```
trait LiftIO[F[_]] {  
  def liftIO[A](ioa: IO[A]): F[A]  
}
```

Scalaz 7 Task

- The name is two characters longer 😓
- Supports all the constructors we want
 - now / delay / async / fail
- Bizarre (and slow!) implementation based on actors
- Still no abstraction (not even LiftIO!)
- Misleading and unsafe concurrency primitives

Scalaz 7 Task

```
val program: Task[Unit] = for {  
  _ <- Task.delay(print("Enter a list of names: "))  
  namesStr <- Task.delay(readLine)  
  
  names = namesStr.split(" ").toList  
  
  _ <- names traverse { name =>  
    Task.delay(println(s"Hello, $name!"))  
  }  
} yield ()  
  
program.unsafePerformSync() // yolo
```

Scalaz 7 Task

```
def readFromChannel(  
  c: AsynchronousSocketChannel): Task[ByteVector] = {  
  
  Task async { cb =>  
    val buf = ByteBuffer.allocate(2048)  
  
    c.read(buf, (), new CompletionHandler {  
      def completed(read: Integer, att: Unit): Unit =  
        cb(V-(ByteVector(buf)))  
  
      def failed(t: Throwable, att: Unit): Unit =  
        cb(-V(t))  
    })  
  }  
}
```

Scalaz 7 Task

```
def writeToChannel(  
  c: AsynchronousSocketChannel,  
  bytes: ByteVector): Task[Unit] = {  
  
  Task async { cb =>  
    val buf = bytes.toByteBuffer  
  
    c.write(buf, (), new CompletionHandler {  
      def completed(written: Integer, att: Unit): Unit =  
        cb(V-(()))  
  
      def failed(t: Throwable, att: Unit): Unit =  
        cb(-V(t))  
    })  
  }  
}
```

Scalaz 7 Task

```
for {  
  // a channel into... the void, I guess  
  c <- Task.delay(AsynchronousSocketChannel.open())  
  
  version <- readFromChannel(c)  
  
  _ <- if (decodeVersion(version) === CurrentV) {  
    for {  
      syn <- readFromChannel(c)  
      _ <- writeToChannel(Ack(syn))  
    } yield ()  
  } else {  
    Task.  
  }  
  
  _ <- Task.delay(c.close())  
} yield ()
```

what thread are we on???

answer: the NIO event dispatcher!



Thread Best Practice

- Divide your thread usage into three categories
 - **Computation**
 - **Blocking IO** (*e.g.* java.io)
 - **Event Dispatchers**
- Allocate one pool for each

Thread Best Practice

- **Computation**
 - Work-stealing non-daemon
 - #cpus in size
- **Blocking IO** (e.g. java.io)
 - Unbounded(!!!) and caching
- **Event Dispatchers**
 - Very small (1-4 threads), high priority, daemon

Scalaz 7 Task

- Gives us almost nothing to solve this problem
 - fork is a trap, don't use it
 - This use-case is *incredibly* common
- Encourages deceptively dangerous parallelism
 - Resource management

```
def both[A, B](ta: Task[A], tb: Task[B]): Task[(A, B)]
```

```
def race[A, B](ta: Task[A], tb: Task[B]): Task[A ∨ B]
```

```
val acquireResource: Task[Rsrc] = ???
```

```
both(acquireResource, Task.fail(new RuntimeException))
```

```
val acquireResource1: Task[Rsrc1] = ???
```

```
val acquireResource2: Task[Rsrc2] = ???
```

```
race(acquireResource1, acquireResource2)
```

Scalaz 7 Task

- Is a step in the right direction
- Doesn't provide sane thread control functionality
- Has extremely broken concurrency features
 - Mostly because of resource management
- Is really, really slow
- Is really, really concrete

Other Options?

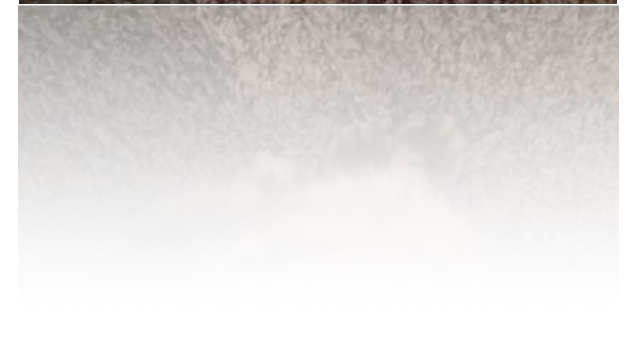
- `ContT[Free[() => ?, ?], Unit, A]`
- fs2 and Monix
 - Resource management
 - Abstraction!
 - Performance
 - Tied to a larger framework

cats-effect



cats-effect

- Defines IO (two fewer characters!) 😊👍
- All the constructors you know and love
 - pure / apply / async / raiseError
- Much simpler and faster internals
- Abstract typeclasses and laws
- *No concurrency!*



cats-effect

```
val program: IO[Unit] = for {  
  _ <- IO(print("Enter a list of names: "))  
  namesStr <- IO(readLine)  
  
  names = namesStr.split(" ").toList  
  
  _ <- names traverse { name =>  
    IO(println(s"Hello, $name!"))  
  }  
} yield ()  
  
program.unsafeRunSync() // yolo
```

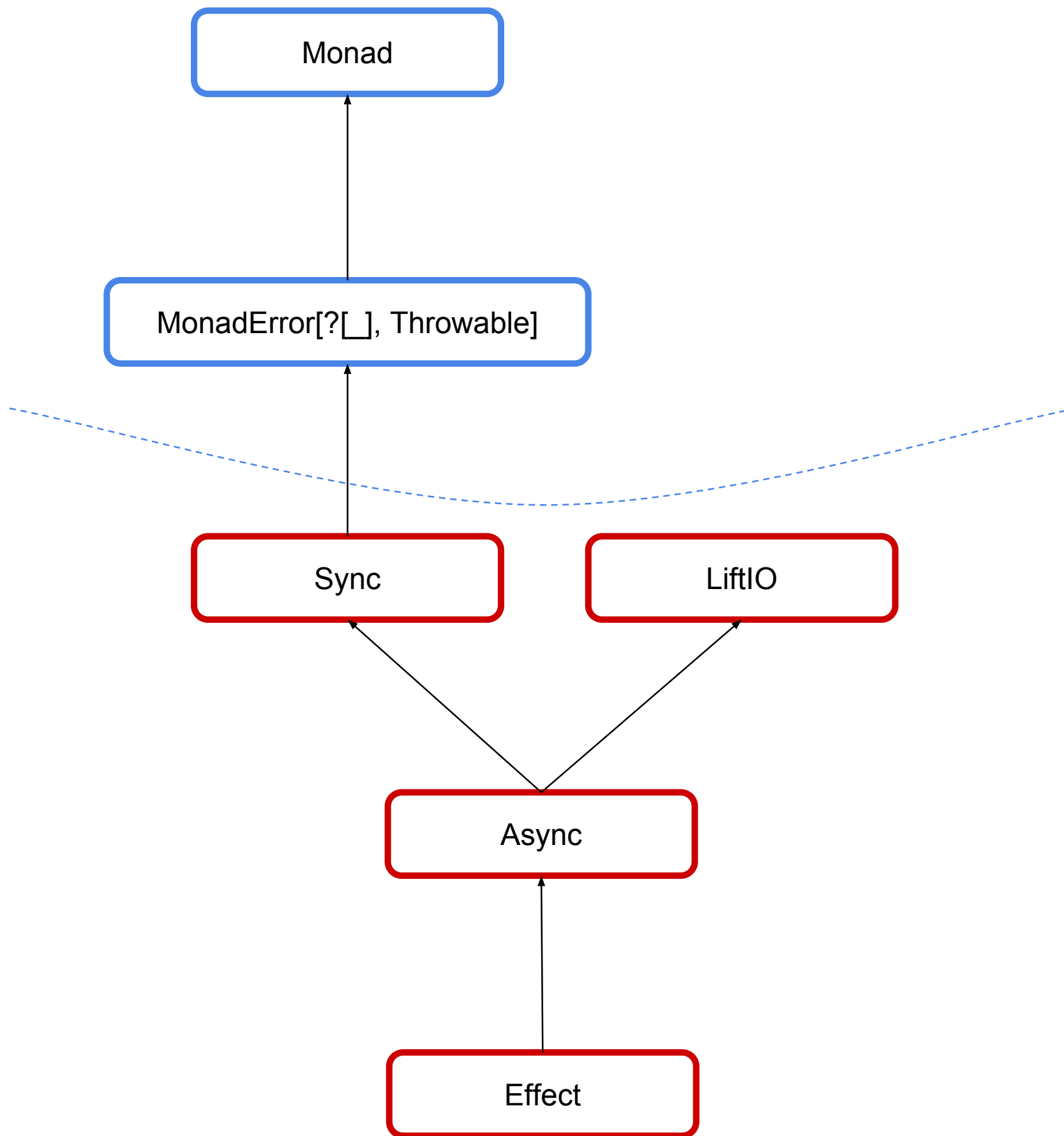
```
for {  
  // a channel into... the void, I guess  
  c <- IO(AsynchronousSocketChannel.open())  
  
  version <- readFromChannel(c)  
  
  _ <- if (decodeVersion(version) === CurrentV) {  
    for {  
      syn <- readFromChannel(c)  
      _ <- writeToChannel(Ack(syn))  
    } yield ()  
  } else {  
    IO.pure()  
  }  
  
  _ <- IO.shift(CPUPool)  
  
  _ <- IO(c.close())  
} yield ()
```

cats-effect

- shift allows thread assignment as an effect
- Relocates computation *following* its sequencing
- The *only* thread-related function
 - Nearly everything “real” uses it
- Not a primitive
 - Baked into the library for convenience

Abstraction





Abstraction

- It's relatively easy to be a Sync
 - Really, just laziness and error handling
- Async requires callback support
- Effect requires you are *literally* equivalent to IO

Abstraction

- Parametricity isn't enough
- It's very easy to build *incorrect* implementations
- We define laws to constrain implementations
- Users can rely on lawful behavior
- Implementors can use laws as a free test suite!


```
trait SyncLaws[F[_]]  
  extends MonadErrorLaws[F, Throwable] {  
  
  implicit def F: Sync[F]  
  
  def delayConstantIsPure[A](a: A) =  
    F.delay(a) <-> F.pure(a)  
  
  def suspendConstantIsPureJoin[A](fa: F[A]) =  
    F.suspend(fa) <-> F.flatten(F.pure(fa))  
  
  def delayThrowsRaiseError[A](e: Throwable) =  
    F.delay[A](throw e) <-> F.raiseError(e)  
  
  // ...  
}
```

```
checkAll("EitherT[Eval, Throwable, ?]",  
  SyncTests[EitherT[Eval, Throwable, ?]].sync[Int, Int, Int])
```

```
[info] - EitherT[Eval, Throwable, ?].sync.monoidal left identity
[info] - EitherT[Eval, Throwable, ?].sync.monoidal right identity
[info] - EitherT[Eval, Throwable, ?].sync.mproduct consistent flatMap
[info] - EitherT[Eval, Throwable, ?].sync.propagate errors through bind (suspend)
[info] - EitherT[Eval, Throwable, ?].sync.repeated sync evaluation not memoized
[info] - EitherT[Eval, Throwable, ?].sync.suspend constant is pure join
[info] - EitherT[Eval, Throwable, ?].sync.tailRecM consistent flatMap
[info] - EitherT[Eval, Throwable, ?].sync.throw in delay is raiseError
[info] - EitherT[Eval, Throwable, ?].sync.throw in suspend is raiseError
[info] - EitherT[Eval, Throwable, ?].sync.unsequenced delay is no-op
```

Abstraction

- Laws prevent invalid instances
- Future
 - Error propagation
 - Laziness
 - Resequencing
- Kleisli
 - Stack safety

Abstraction

- Frameworks build against the abstractions
 - fs2
 - http4s
 - doobie
 - ...and more!
- You aren't forced to use *my* implementation of IO

Conclusion

- Effects are scary (in *all* languages)
- The IO monad is a really useful thing!
 - Especially if we can be generic about it
- Pre-existing options were all bad
 - Though pioneering in their time!
- cats-effect is a thing



